

Лекция 1. Изображение как сигнал и как данные

Формирование, сенсор, массив чисел, форматы

Александр Савельев · Курс «Анализ изображений», 3 курс бакалавриата

Часть 1

О курсе

Этот семестр: «Анализ изображений»

- Изображение как сигнал: фильтрация, частоты, пирамиды
- Структура: границы, особые точки, геометрия, сегментация, морфология
- Признаки и классическое распознавание
- Обратные задачи: восстановление

Без нейросетей. Всё, что делаем, — понимаем до последней формулы.

Следующий семестр: «Глубокое обучение в задачах КЗ»

- Свёрточные сети, детекторы, сегментация, генерация
- Использует **наш** язык: свёртка, пирамида, IoU, NMS, PSNR/SSIM, гомография

В конце каждой лекции — слайд **«Мост»**: где эта классическая идея живёт в современных сетях.

13 лекций × 90 минут (этот курс — теория):

- ~10 мин — повторение и мотивация задачи
- ~40 мин — модели и формулы: что означает каждая буква
- ~35 мин — алгоритмы, разбор примеров на картинках
- ~5 мин — «мост» к глубокому обучению, итоги

Практики — отдельные занятия с другим преподавателем: там код (Python, NumPy, OpenCV, scikit-image) и домашние задания.

Оценка

- Домашние задания практик — **допуск** к экзамену
- Экзамен по лекциям — устный



Пять сквозных линий — будем возвращаться к ним каждую лекцию

1. Изображение — функция $I : \Omega \subset \mathbb{R}^2 \rightarrow \mathbb{R}^C$; методы — операторы над ней или оптимизация по ней.
2. Локальность и масштаб: что «видно», зависит от размера окрестности.
3. Инвариантность (сдвиг, поворот, масштаб, свет) — и её цена.
4. Прямая задача (фильтр, детектор) vs обратная (восстановление, регистрация).
5. У каждой задачи своя метрика — разбираем, что она измеряет и где ломается.

Одна фраза

Изображение — это результат цепочки **свет** → **оптика** → **сенсор** → **массив чисел** → **файл**. Все алгоритмы курса ломаются, если не понимать, что именно стоит в пикселях.

Три идеи, ради которых читается вся лекция

1. Пиксель — не «яркость точки», а **среднее по площадке** за время выдержки.
2. Проекция $x = fX/Z$ **нелинейна** по глубине; из одного снимка глубину не восстановить.
3. В компьютере картинка — **тензор** $H \times W \times C$: цвет посчитан (Байер), коды уже с гаммой, JPEG уже сжат.

В конце лекции вернёмся сюда — проверим, что каждая идея легла на своё место.

- **Медицина:** рентген, КТ, МРТ, гистология — измерить, найти, сравнить
- **Документы и биометрия:** распознать паспорт, сравнить лицо с фото, отличить живого человека от фотографии
- **Промышленность:** дефекты на конвейере, измерения, контроль сборки
- **Дистанционное зондирование:** спутники, дроны, сельское хозяйство
- **Наука:** микроскопия, астрономия, физика частиц

Три уровня работы с изображением

Обработка изображение → изображение
фильтрация, сжатие, восстановление

Анализ изображение → измерения, области, признаки
сегментация, дескрипторы, геометрия

Зрение изображение → понимание сцены
«что это и где», 3D, действия

Курс — в основном второй уровень, с опорой на первый.

Часть 2

Формирование изображения

Определение

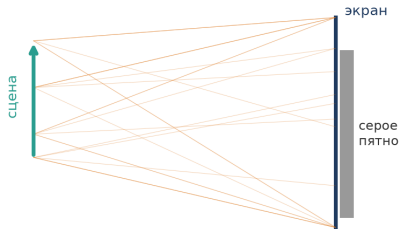
Камера-обскура (лат. *camera obscura* — «тёмная комната») — самая простая возможная камера: **тёмный ящик с маленьким отверстием** в одной из стенок. На противоположной стенке (экране) появляется перевёрнутое изображение внешнего мира.

Зачем она нам нужна

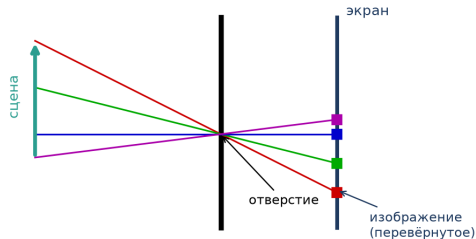
- Это **базовая модель любой камеры** — телефон, зеркалка, камера в машине. Сложная оптика — это лишь «улучшение» обскуры (вместо дырки — линза).
- Она объясняет главный вопрос: **почему вообще получается изображение**, а не световая каша.
- Из неё мы **выведем формулу проекции** $x = fX/Z$ — основу всей геометрической части курса.

Историческая справка: явление описал ещё Мо-цзы (V в. до н.э.), математику — аль-Хайсам (X век).

Без отверстия: свет от каждой точки попадает на весь экран → серое пятно

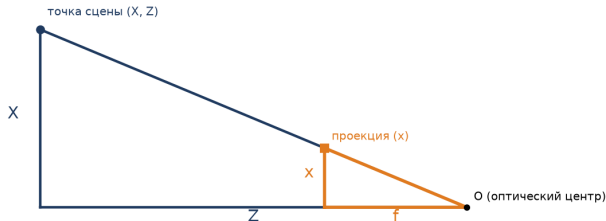


С маленьким отверстием: от каждой точки проходит один луч → получается изображение



Слева: каждая точка сцены светит **во все стороны**. Лучи от всех точек падают на всю поверхность экрана и перемешиваются — серое пятно. **Справа:** маленькое отверстие **пропускает только один луч** от каждой точки сцены — каждая точка сцены попадает в свою точку на экране. Получается изображение — перевернутое, потому что лучи пересекаются в отверстии.

Подобие треугольников: $\frac{x}{f} = \frac{X}{Z} \Rightarrow x = f \frac{X}{Z}$



Два подобных треугольника с общей вершиной в O:

$$\frac{x}{f} = \frac{X}{Z} \Rightarrow x = f \frac{X}{Z}$$

Вся геометрия камеры дальше строится вокруг этих подобных треугольников. Дифференцировать не нужно — достаточно школьного подобия.

Точка сцены (X, Y, Z) в системе координат камеры отображается в точку плоскости изображения:

$$x = f \frac{X}{Z}, \quad y = f \frac{Y}{Z}$$

- f — фокусное расстояние (расстояние от отверстия до плоскости изображения);
- Z — глубина: расстояние вдоль оптической оси;
- (x, y) — координаты на изображении (в тех же единицах, что f).

Вывод — из подобия треугольников на предыдущем слайде: $\frac{x}{f} = \frac{X}{Z}$.

Что из этого следует

- Деление на $Z \Rightarrow$ далёкое — меньше. Проекция **нелинейна** по Z .
- Параллельные прямые в сцене сходятся в **точке схода**.
- Из одной картинке Z не восстановить: вся прямая $(\lambda X, \lambda Y, \lambda Z)$ даёт один и тот же пиксель.
- При $Z \gg$ размера объекта проекция $\approx$ масштабирование — этим пользуются, когда «забывают» о перспективе.

В лекции 7 запишем это в однородных координатах и получим гомографию.

Человек ростом $X = 1,8$ м, объектив $f = 50$ мм $= 0,05$ м. Считаем $x = fX/Z$:

Z	$x = fX/Z$ на сенсоре	относительно $Z=8$ м	что сделали с Z
1 м	90 мм	$\times 8$	в 8 раз ближе
2 м	45 мм	$\times 4$	в 4 раза ближе
4 м	22,5 мм	$\times 2$	в 2 раза ближе
8 м	11,25 мм	$\times 1$	как есть

Ключ

Удвоение расстояния ровно вдвое уменьшает проекцию. $1 \rightarrow 2$, $2 \rightarrow 4$, $4 \rightarrow 8$ — каждый раз x падает вдвое, хотя «метры» разные: $+1$, $+2$, $+4$.

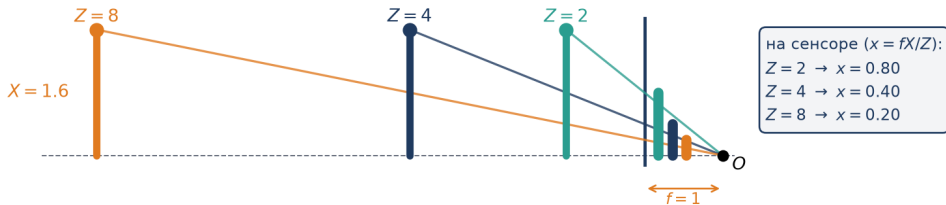
Формула, которая это кодирует

$$\frac{x_1}{x_2} = \frac{Z_2}{Z_1}$$

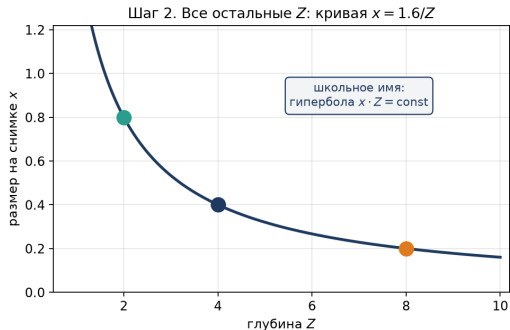
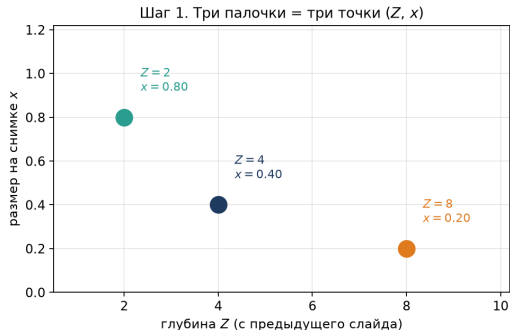
Отношение размеров на снимке — это **обратное** отношение глубин. Не разность метров.

Один рост, три глубины: подобие само рисует $1/Z$ Формирование изображения

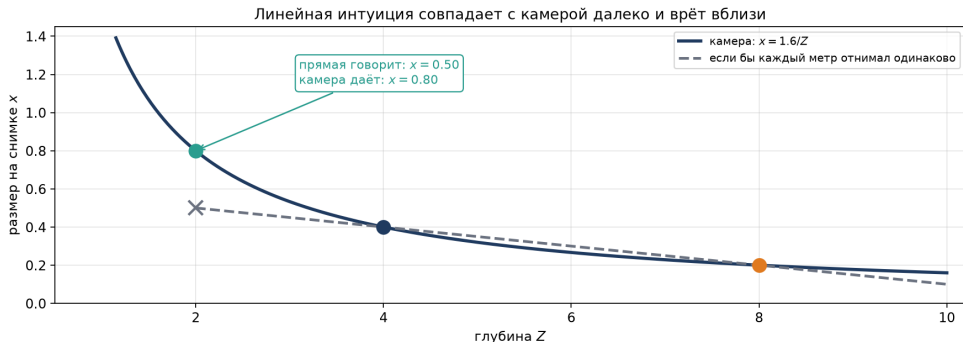
Один рост X , один фокус f : удвоение Z делит проекцию пополам



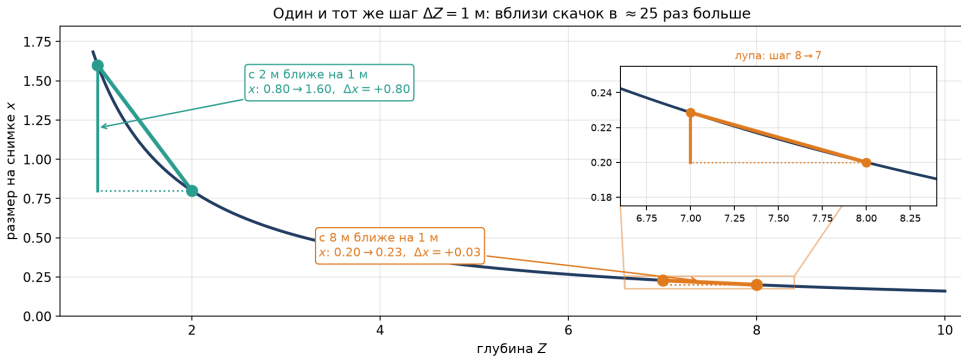
X и f **одни и те же**. Растёт только Z — и маленький треугольник становится всё ниже:
 $x = 0,80 \rightarrow 0,40 \rightarrow 0,20$. Это и есть $x \propto 1/Z$.



На предыдущем слайде у каждой палочки была пара чисел (Z, x). Слева просто отложили их: (2; 0,80), (4; 0,40), (8; 0,20). Справа добавили все остальные Z — получилась кривая $x = 1,6/Z$.
Её школьное имя — **гипербола**: произведение $x \cdot Z$ постоянно (у нас 1,6).



Интуиция «каждый метр ближе — плюс столько-то пикселей» была бы **прямой**. Прямая, подобранная по дальним точкам $Z=4$ и $Z=8$, вблизи врёт: при $Z=2$ она обещает $x=0,50$, камера даёт $x=0,80$.



Берём **один и тот же** шаг $\Delta Z = 1$ м — но в двух местах гиперболы. Издали картинка почти не меняется ($\Delta x = 0,03$); вблизи — подскакивает ($\Delta x = 0,80$). Крутизна графика зависит от того, **где** вы стоите, а не только от длины шага.

С предыдущего слайда: одинаковый ΔZ даёт разный Δx . Для маленького шага это кодируется так:

$$\frac{dx}{dZ} = -\frac{fX}{Z^2} \implies \frac{\Delta x}{x} \approx -\frac{\Delta Z}{Z}$$

- dx/dZ — крутизна графика $x(Z)$: насколько изменится снимок, если сдвинуться на метр;
- минус: отошли дальше $\Rightarrow$ стали меньше;
- удобная форма: **относительное** изменение размера $\approx$ относительный сдвиг $\Delta Z/Z$.

Портрет, $Z \approx 2$ м

Нос торчит на 12 см вперёд.

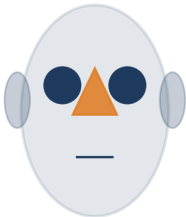
$\Delta Z/Z = 0,12/2 = 6\%$. Все части лица масштабируются почти одинаково.

Селфи, $Z \approx 0,3$ м

Тот же нос: $\Delta Z/Z = 0,12/0,3 = 40\%$.

Ближняя часть на 40% крупнее дальней — нос раздувается. Это **тот же принцип**, что на предыдущем слайде: важен $\Delta Z/Z$.

Портрет: лицо в 2 м
нос ближе на 12 см = 6% глубины



нос/ширина лица на снимке: 30 %
в жизни: 29 %
 $Z_{\text{нос}} = 1.88 \text{ м}$, $Z_{\text{лицо}} = 2.00 \text{ м}$

Селфи: лицо в 0.3 м
нос ближе на 12 см = 40% глубины



нос/ширина лица на снимке: 48 %
в жизни: 29 %
 $Z_{\text{нос}} = 0.18 \text{ м}$, $Z_{\text{лицо}} = 0.30 \text{ м}$

Картинки нормированы по овалу лица, чтобы сравнить **пропорции**. На селфи нос занимает заметно большую долю ширины, чем в жизни: он ближе, а $x \propto 1/Z$.

Можно

- Один и тот же **малый** объект: в 2 раза больше на снимке $\Leftrightarrow$ в 2 раза ближе (Z уменьшился вдвое).
- Отношение глубин двух объектов **известного** размера:
$$Z_2/Z_1 = (X_2/X_1) \cdot (x_1/x_2).$$

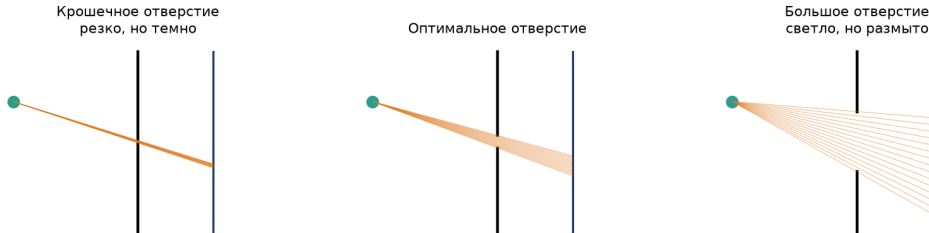
Когда почти линейно

Если $Z \gg$ размера объекта, все его точки делятся почти на одно и то же Z — проекция $\approx$ масштабирование. Так «забывают» перспективу в аэрофотосъёмке.

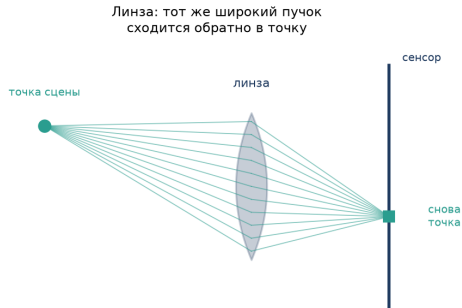
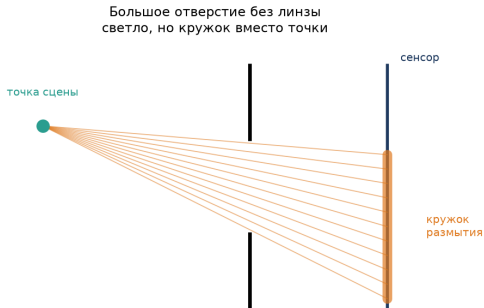
Нельзя

- Сравнить объекты **неизвестного** размера: большой далёкий и маленький близкий выглядят одинаково.
- Считать 3D-объект «похожей копией»: разные части на разном Z масштабируются по-разному.
- Переводить пиксели в метры без f и Z (нужна калибровка).

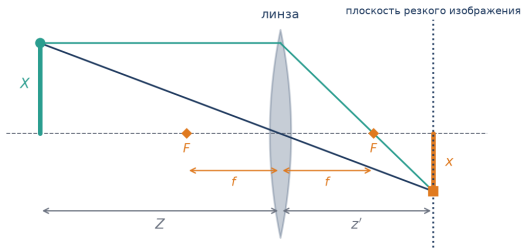
Частая ошибка: «подошёл на 2 метра — стало в 2 раза больше». Нет: 2 м при $Z=10$ м это 20%, при $Z=3$ м — уже 67%.



Каждая точка сцены светит во все стороны. Через маленькое отверстие проходит **тонкий** пучок — на сенсоре почти точка (резко), но фотонов мало (темно). Через большое — **широкий** пучок: света много, но он размывается в кружок.



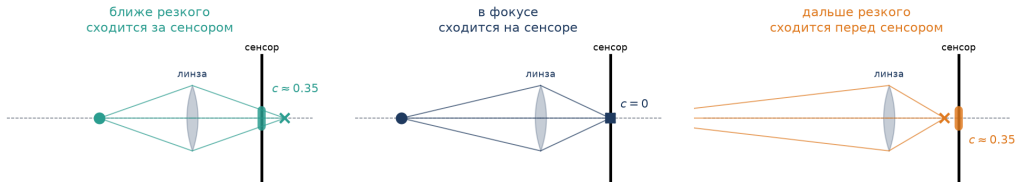
Линза **загибает** края широкого пучка так, что они снова встречаются в одной точке. Получаем и свет большого отверстия, и резкость маленького. Плата: сходятся только лучи от точек на **одном** расстоянии Z . Остальные — снова кружок.



Два луча: через центр и параллельный оси (в F). Где встретились — резкое изображение.

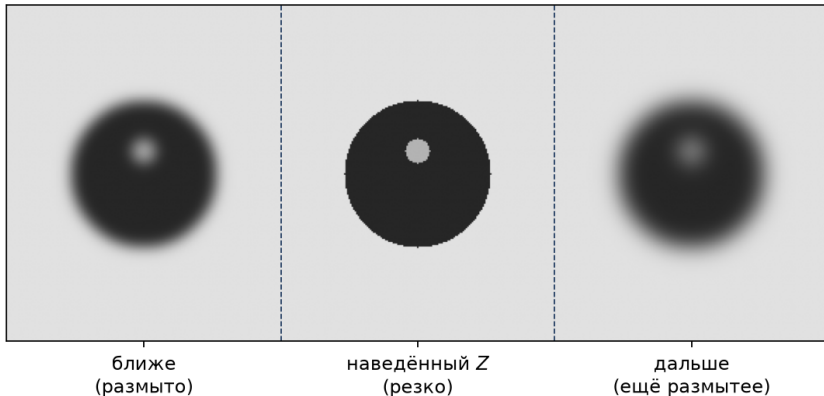
$$\frac{1}{Z} + \frac{1}{z'} = \frac{1}{f}$$

- Z — до объекта; z' — до резкого изображения; f — фокус (лучи с бесконечности).

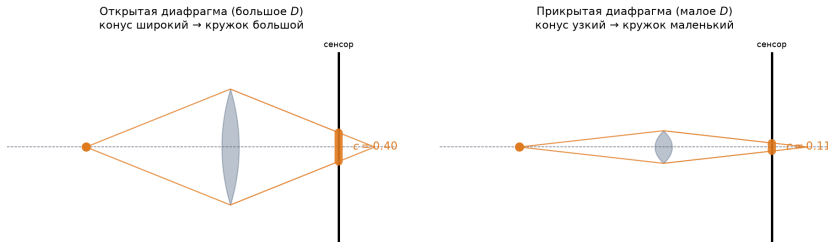


Мы наводим резкость, двигая линзу, пока z' нужного Z не совпадёт с сенсором. Для любого другого Z пучок пересекает сенсор **не в точке**, а диском — *circle of confusion* c . Если c меньше пикселя, глаз и алгоритм считают объект резким.

Глубина резкости: резко только узкий диапазон Z , остальное — кружок больше пикселя



Глубина резкости (DoF) — диапазон Z , внутри которого кружок меньше пикселя. Всё, что ближе и дальше, выглядит мягким. Это не «сломанная» камера — это геометрия конуса лучей.



Тот же промах по Z , разный диаметр D . Узкий конус $\Rightarrow$ маленький кружок на сенсоре.

DoF растёт, если

уменьшить D (число $N = f/D$ больше); взять меньшее f ;
отойти дальше.

Расфокус позже = свёртка с диском
(лекция 3).

Пиксель — не «цвет поверхности», а **сколько света накопилось** за выдержку:

$$I \propto \underbrace{\rho}_{\text{альбедо}} \cdot \underbrace{E_{\text{src}}}_{\text{свет на поверхности}} \cdot \underbrace{\left(\frac{D}{f}\right)^2}_{\text{насколько открыта линза}} \cdot \underbrace{t}_{\text{выдержка}}$$

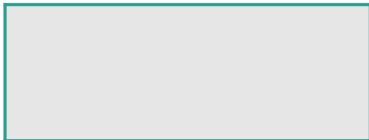
- ρ — какая доля падающего света отражается: белая бумага $\sim 0,8$, уголь $\sim 0,05$;
- E_{src} — насколько ярко освещена эта точка сцены;
- D/f — относительное отверстие (то же D , что в глубине резкости): шире — больше света;
- t — время, пока затвор открыт.

Четыре ручки, одно число

Открыли диафрагму — и светлее, и меньше DoF. Удлинили t — светлее, но если что-то движется, размажется.

Пиксель $I \propto \rho \cdot E$: белое в тени может совпасть с серым на свету

белая бумага, яркий свет
 $\rho = 0.90, E = 1.00 \Rightarrow I = 0.90$



белая бумага, тень
 $\rho = 0.90, E = 0.50 \Rightarrow I = 0.45$



серая бумага, яркий свет
 $\rho = 0.45, E = 1.00 \Rightarrow I = 0.45$

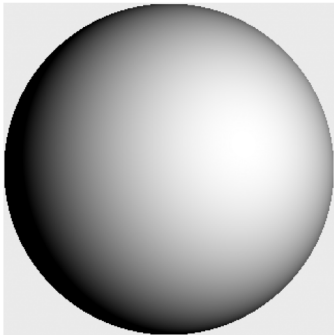


серая бумага, тень
 $\rho = 0.45, E = 0.50 \Rightarrow I = 0.23$



Белая бумага в тени и серая на свету дают **одно и то же** $I = 0,45$. По одному снимку нельзя сказать, светлая это поверхность в тени или тёмная на солнце. Отсюда тени, которые алгоритмы путают с объектом, и баланс белого.

Матовая сфера: ярче там,
где нормаль смотрит на свет



$$L \propto \rho \max(0, \mathbf{n} \cdot \mathbf{l})$$

чем меньше угол между $\mathbf{n}$ и $\mathbf{l}$,
тем ярче (косинус).



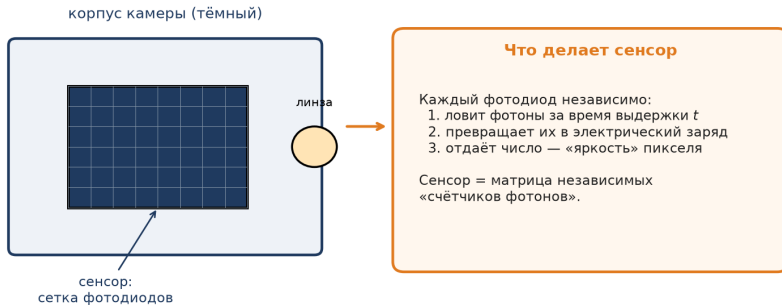
$\mathbf{n}$ — нормаль («куда смотрит» площадка), $\mathbf{l}$ — направление на источник. Матовая поверхность не зависит от того, откуда смотрит камера: повернули наблюдателя — яркость та же. Блики (зеркало, лак) этой моделью **не** описываются: они едут вместе с камерой.

Часть 3

Сенсор и конвейер камеры

Определение

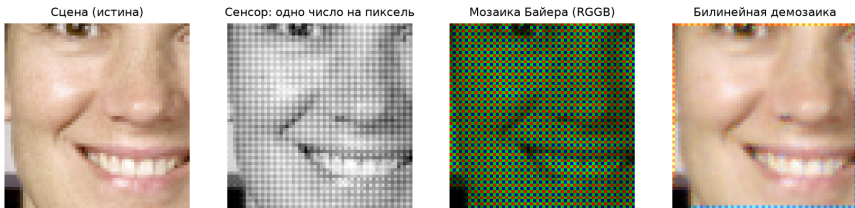
Сенсор (англ. *image sensor*) — электронный чип, который **превращает свет в числа**. Состоит из матрицы **фотодиодов** — маленьких светочувствительных ячеек. Один фотодиод даёт один пиксель.





ISO усиливает g — умножает и сигнал, и шум

1. **Фотоны прилетают случайно.** Свет — поток частиц. За выдержку t в пиксель попадает **случайное число** фотонов N .
2. **Фотон → электрон.** Не каждый фотон выбивает электрон — доля QE (Quantum Efficiency) $\approx 0.5\text{--}0.8$.
3. **Заряд накапливается** в «колодце» (full-well); переполнение $\Rightarrow$ пересвет.
4. **Заряд → напряжение V :** $V = g \cdot e$, $g = \text{ISO}$. Выше ISO — больше и сигнал, и шум.
5. **$V \rightarrow$ целое I — АЦП** (аналого-цифровой преобразователь). Здесь — ошибка квантования.



Фотодиод считает фотоны, **не спрашивая длину волны**. Чтобы получить цвет, перед каждой ячейкой ставят светофильтр: пропускает только узкую полосу. На выходе сенсора — **одно** число на пиксель, не тройка (R, G, B) .

Почему зелёных вдвое больше

Сенсор и конвейер камеры

сцена



только R



только G $\approx$ ч/б фото



только B — другой снимок



яркость $Y \approx 0.3R + 0.6G + 0.1B$



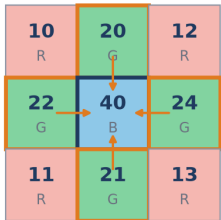
плитка 2 x 2: зелёных вдвое больше

R	G	R	G
G	B	G	B
R	G	R	G
G	B	G	B

Края читаем по яркости, а яркость почти зелёная. Поэтому в мозаике зелёных **вдвое больше** — цвет можно проредить.

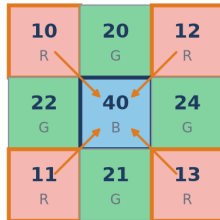
Центр измерил только синий $B = 40$. Остальные каналы — среднее соседей.

Нужен зелёный: 4 соседа крестом



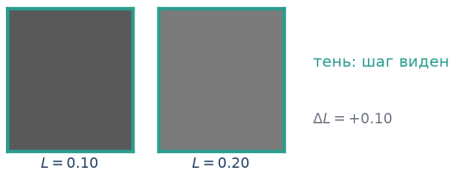
$$\hat{G} = (20 + 22 + 24 + 21)/4 = 21,75$$

Нужен красный: 4 соседа по углам

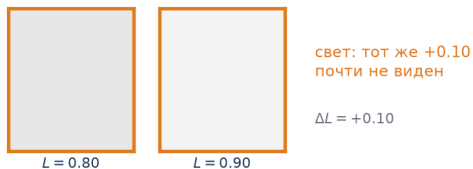
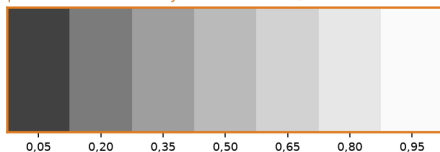


$$\hat{R} = (10 + 12 + 11 + 13)/4 = 11,5$$

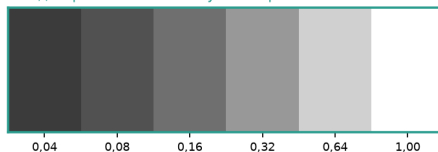
Недостающий канал = среднее ближайших ячеек с нужным фильтром. В цветном снимке **2 из 3** чисел посчитаны. На цветном крае среднее врёт $\Rightarrow$ кайма.



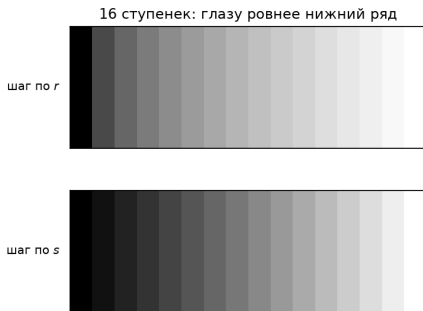
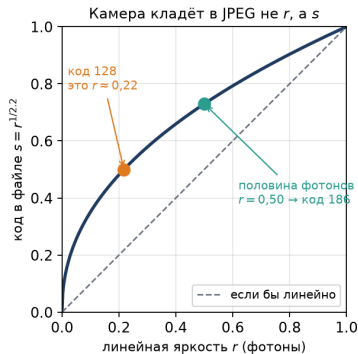
равные +0.15 по свету: в тенях скачки, в светах каша



каждый раз $\times 2$: шаги глаза почти равные

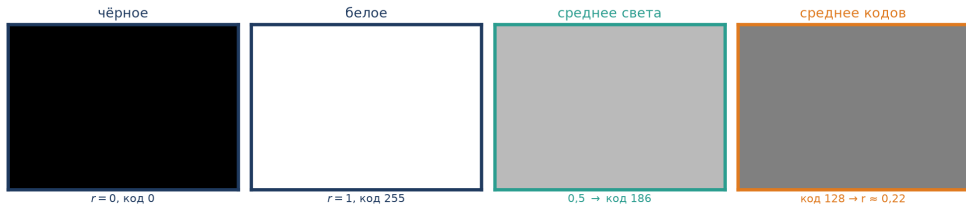


Едва заметный шаг пропорционален самой яркости: $\Delta L/L \approx \text{const.}$ «Одинаково для глаза» — это **умножить** свет (1, 2, 4, 8, ...), а не прибавить одно и то же. Стивенс (упрощение): субъективно $\approx L^{0.3-0.5}$; камеры берут степень 1/2, 2.



$s = r^{1/2.2}$, $r = s^{2.2}$, в 8 битах пиксель = $\text{round}(255 s)$. Код 128 — это $\approx 22\%$ света; половина фотонов даёт код 186.

Чёрное + белое: среднее фотонов светлее, чем среднее JPEG-кодов



Нужна физика — сначала $r = (\text{код}/255)^{2.2}$, считаем, на экран обратно $s = r^{1/2.2}$. Границы по кодам sRGB считать можно; числа как яркость в физике — нельзя.

Часть 4

Изображение как массив

фотография



та же картинка — таблица 8×8

56	107	112	111	199	175	176	190
110	109	98	152	177	150	140	147
98	96	85	157	154	94	99	106
117	88	77	109	168	143	158	158
113	77	78	74	140	194	167	165
119	77	93	85	64	82	137	144
113	88	75	49	36	44	29	34
87	95	58	33	57	64	14	17

Каждый квадратик — **пиксель**. Число в клетке — яркость (0...255). Цвет появится на следующем слайде: там уже три числа.

Пиксель в рамке: $(R, G, B) = (217, 123, 41)$

цвет (R, G, B)



канал R



канал G



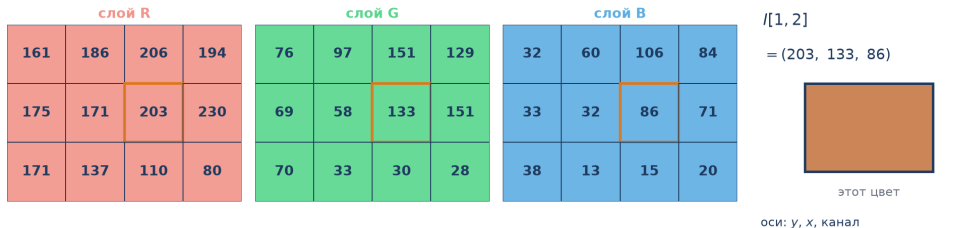
канал B



Цветной пиксель — тройка (R, G, B) : сколько красного, зелёного и синего. Три канала — это три ч/б картинки; вместе они дают цвет.

Три канала — это тензор $H \times W \times 3$

Изображение как массив



Форма массива: **высота** $\times$ **ширина** $\times$ **каналы**. Первая ось — строка (y), вторая — столбец (x).
Не наоборот.

RGB: 3 числа



серое: 1 число



таблица яркости

56	107	112	111	199	175
110	109	98	152	177	150
98	96	85	157	154	94
117	88	77	109	168	143
113	77	78	74	140	194
119	77	93	85	64	82

Серое — тензор $H \times W$ (канал один). Часто берут яркость $Y \approx 0,3 R + 0,6 G + 0,1 B$ — ту же, что на слайде про Байер.

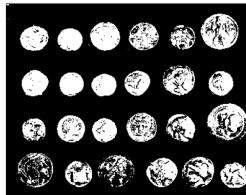
RGB
 $H \times W \times 3$



серое
 $H \times W$



маска 0/1
объект или нет



RGBA
4-й канал — прозрачность



Маска: каждый пиксель 0 или 1 — «это объект или фон». **RGBA:** четвёртый канал — прозрачность; под картинкой виден фон.

Часть 5

Форматы файлов

Без потерь

PNG — 8/16 бит, альфа-канал, сжатие DEFLATE (~2–3× на фото, огромное — на графике).

TIFF — контейнер: многостраничный, 16/32 бит, float; стандарт в медицине и научной съёмке.

BMP/PPM — без сжатия, для отладки.

Специализированные

DICOM (медицина: пиксели + метаданные пациента и геометрии), NIfTI (3D-объёмы), GeoTIFF (спутники), RAW-форматы камер (DNG, CR2, NEF).

С потерями

JPEG — 8 бит, YCbCr, блоки 8×8 , DCT + квантование (лекция 4); 10–20× при «незаметных» потерях.

WebP / AVIF / HEIC — лучше на тех же байтах.

Правила для анализа

- Промежуточные результаты — **только PNG/TIFF/NumPy**: каждое пересохранение в JPEG добавляет ошибки.
- Артефакты JPEG — отдельный «шум» и источник **domain shift**.
- EXIF: ориентация хранится **флагом** (поворот на 90° !), выдержка, ISO, модель камеры.

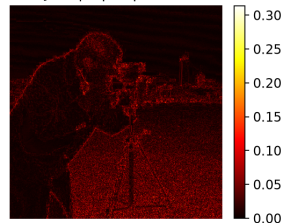
«RAW»: линейный, 16 бит
годится для физики



JPEG q=30: гамма+8 бит+DCT
годится для просмотра



|RAW - JPEG|: артефакты сжатия



RAW: линейный, 12–14 бит — для физики. **JPEG**: гамма, 8 бит, DCT + chroma-subsampling — для просмотра.
Промежуточные — PNG/TIFF/NumPy. Каждое пересохранение JPEG **добавляет** ошибок.

Часть 6

Итоги

Где эта идея живёт в глубоком обучении

- Нейросеть получает **ровно тот тензор** $H \times W \times C$: те же оси, та же нормализация в $[0, 1]$.
- JPEG, гамма, баланс белого и демозаика разных камер — главные источники **domain shift**: модель с iPhone теряет качество на другом телефоне.
- Обучаемые ISP (демозаика, HDR) заменяют блоки конвейера камеры — постановка задачи та же.

Подробно — в курсе «Глубокое обучение в задачах компьютерного зрения». Здесь только связь, без деталей.

1. Изображение — результат **проекции** $x = fX/Z$. В компьютере это **тензор** $H \times W \times C$.
2. Значение пикселя $\propto$ освещение $\times$ отражательная способность — разделить их по одному снимку нельзя.
3. Сенсор даёт одно число на пиксель; цвет — демозаика. В цветном снимке **2 из 3** чисел посчитаны.
4. RGB — три числа, серое — одно, маска — 0/1. Первая ось — строка (y).
5. Код 128 — не половина света. Физику считаем в линейном: $r = (\text{код}/255)^{2.2}$.
6. RAW линейен; JPEG уже с гаммой и сжатием. Помним EXIF-ориентацию.

Литература

Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., гл. 2 — <https://szeliski.org/Book/>.

Gonzalez, Woods, *Digital Image Processing*, 4th ed., гл. 2 (Digital image fundamentals).

1. Выведите $x = fX/Z$. Почему $x(Z)$ — гипербола? Почему по одному снимку нельзя восстановить Z ?
2. Почему уменьшение отверстия камеры-обскуры делает изображение резче, и что ограничивает этот эффект?
3. Что измеряет один пиксель сенсора с фильтром Байера? Какая доля значений цветного снимка — интерполяция?
4. Зачем нужна гамма-коррекция, и в каком пространстве нужно усреднять яркости?
5. Чем RGB-картинка отличается от серой как тензор? Что лежит в $I[y, x]$, и какая ось первая?
6. Почему в мозаике Байера зелёных вдвое больше, а не поровну R, G, B ?
7. Как билинейная демозаика заполняет недостающий канал?
8. Почему код 128 — не половина света? Как правильно усреднить чёрное и белое?
9. Чем маска отличается от RGBA? Какая форма у RGB и у серого массива?
10. Чем RAW отличается от JPEG как *данные* для анализа? Назовите три следствия.